

manulator

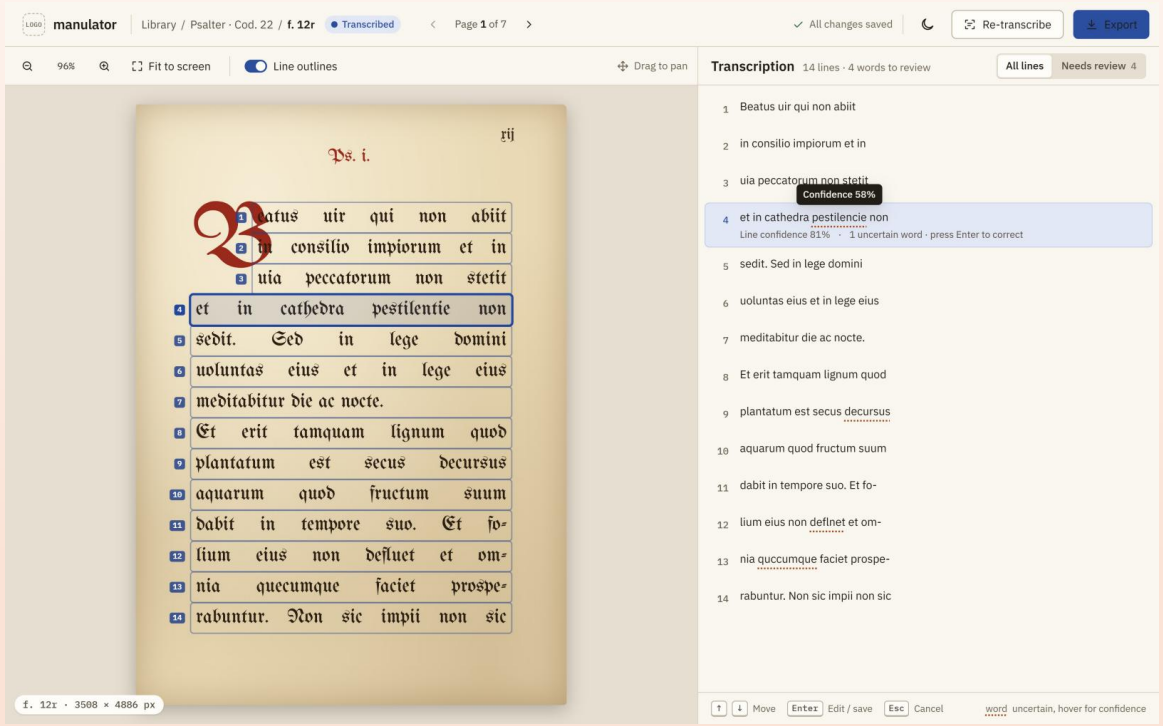
An interactive library for historical manuscripts

Transcribe, investigate and share

Advanced Software Engineering / Mid-Term Team Presentation

Product vision: an interactive library

Specialized recognition, visual verification and a reusable workspace for medieval historians



Intended final experience / existing project interface design

A focused starting corpus

Latin charters, Western Europe, 1200–1400

Beyond a general LLM chat

One recognizer suited to this initial corpus

Check each reading

Compare scan and text, then correct errors

Keep a research library

Save manuscripts and return to your work

Planned research tools

English translation, summaries and sharing

PART 01 / 05

Demonstration

The working software in action

manulator

Read medieval manuscripts, line by line.

Sign in

Welcome back. Pick up where you left off.

Email

Password

Show

Forgot password?

Sign in

New to manulator? Create an account

Read medieval manuscripts, line by line.

pload a page scan and get a transcription you can correct, search and export.

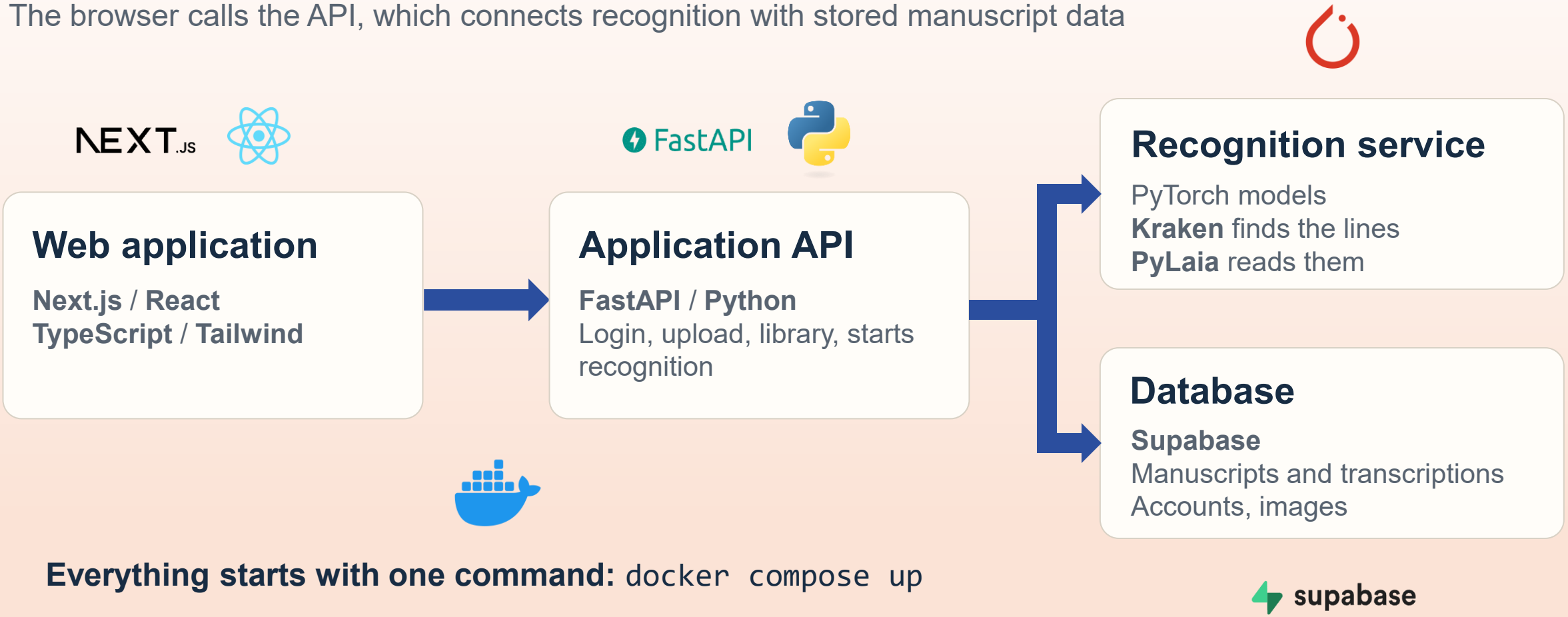
PART 02 / 05

Technology stack

Main components and recognition workflow

How the application fits together

The browser calls the API, which connects recognition with stored manuscript data



Everything starts with one command: `docker compose up`

How the repository is organized

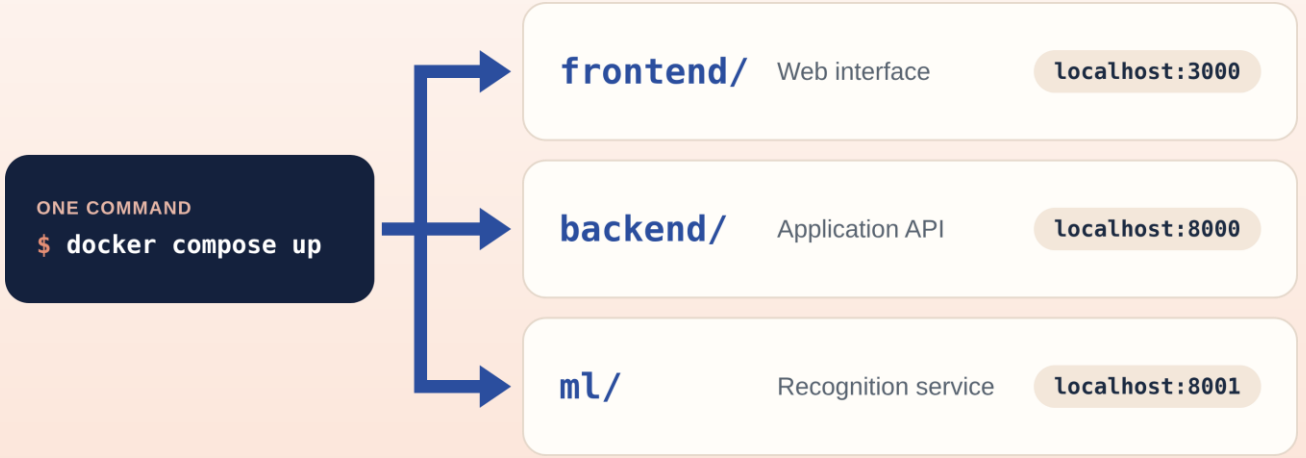
The README explains the project structure and how to run each part

`frontend/`
Web interface

`backend/`
API and application services

`ml/`
Recognition service (Kraken, PyLaia)

`docs/`
Architecture and documentation



README.md: setup, Docker and team workflow

How our recognition models work

1 Kraken

Finds the text lines
and keeps their positions

2 PyLaia

Recognises the handwriting
one line at a time

Why we chose these models

Kraken

Pretrained line detection
ready to use

PyLaia

Expands abbreviations into full words
Lowest character error rate in our tests
Fast transcription

PART 03 / 05

User stories

How we manage requirements and define done

How we manage user stories

A user need becomes a GitHub issue with clear acceptance criteria

1

User story

Describe the need and its purpose

2

Tasks

Break the story into work we can assign

3

Pull request

Implement the change on a feature branch

4

Review & merge

Check the criteria, tests and review

Our definition of done

Criteria checked and CI passing
A teammate reviews the change before dev

A user story in GitHub

Issue #80: a medieval researcher checks the transcription against the original scan

User can view transcription result beside the original document

Need

As a medieval researcher **I want** to see the transcribed text side by side with the uploaded manuscript scan with the detected **so that** I can compare each line against the original handwriting and can easily see which line was detected and spot transcription errors before correcting them.

Reason

HTR output (Kraken + PyLaia) is never perfect on medieval hands. Without seeing the scan next to the text, users can't judge whether a line is right, and the later correction and translation steps rest on unchecked text. This view is the bridge between "upload → transcribe" (minimal functionality) and "correct → translate".

The story’s acceptance criteria

Issue #80 defines the expected view, line order and failure behaviour

Acceptance Criteria

#	Acceptance Criteria (action → expected result)	Status (Won't do / Problem / Done)
AC1	Open a document whose transcription is finished → scan shown on the left, transcription on the right	
AC2	Look at the transcription pane → text appears line by line in the same order as on the scan	
AC3	Open a document whose transcription failed → error message shown with no crash; scan still visible	

Researcher’s goal

Check each line against the scan
before correcting or translating the text

What we verify

AC1 Scan and text together

AC2 Same reading order

AC3 Clear failure message

Done requires

CI, review and merge

PART 04 / 05

Continuous integration

What runs before and after a merge

What happens when a feature is merged?

GitHub Actions checks pull requests and runs again after a push to dev or main

01

Feature branch

Work on one task

02

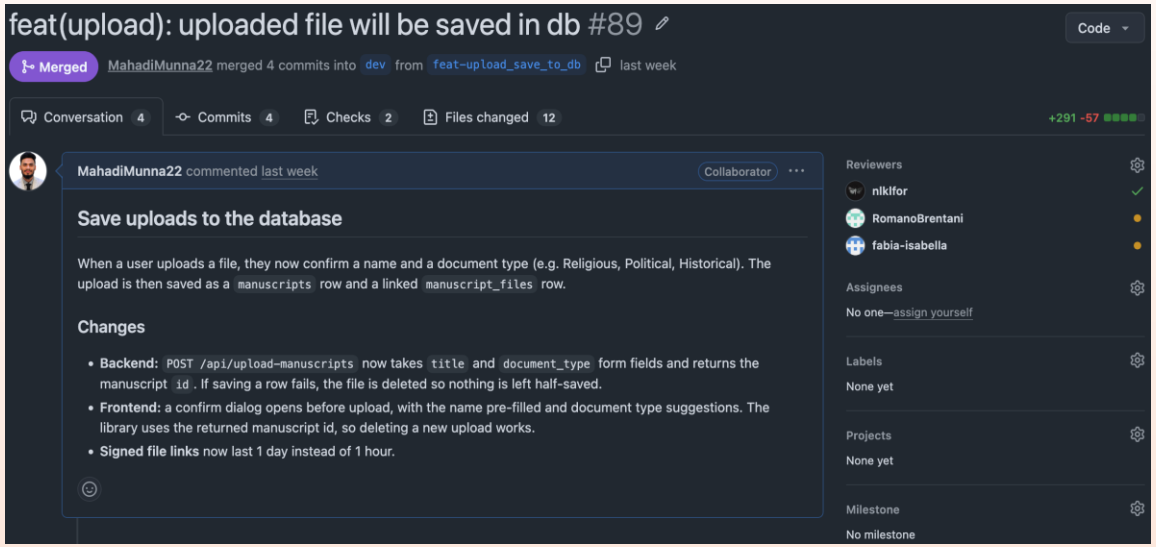
Pull request

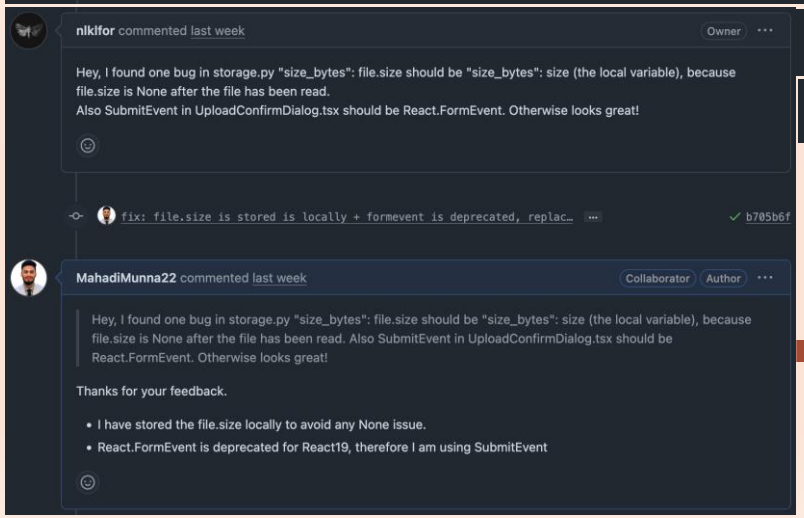
Open against dev

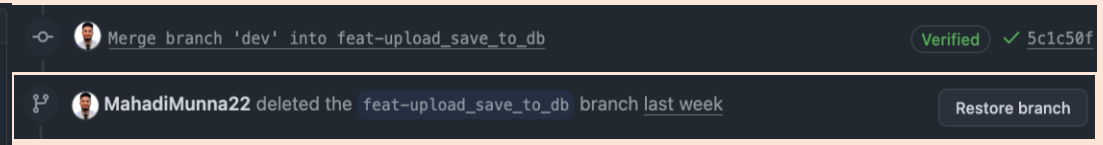
03

Checks & review

CI + one approval







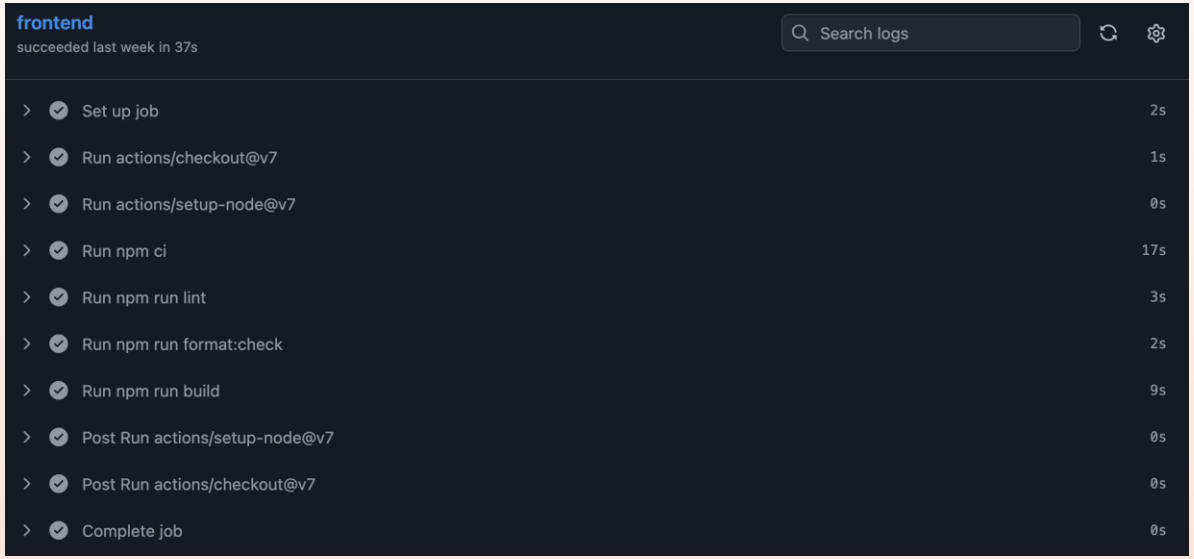
04

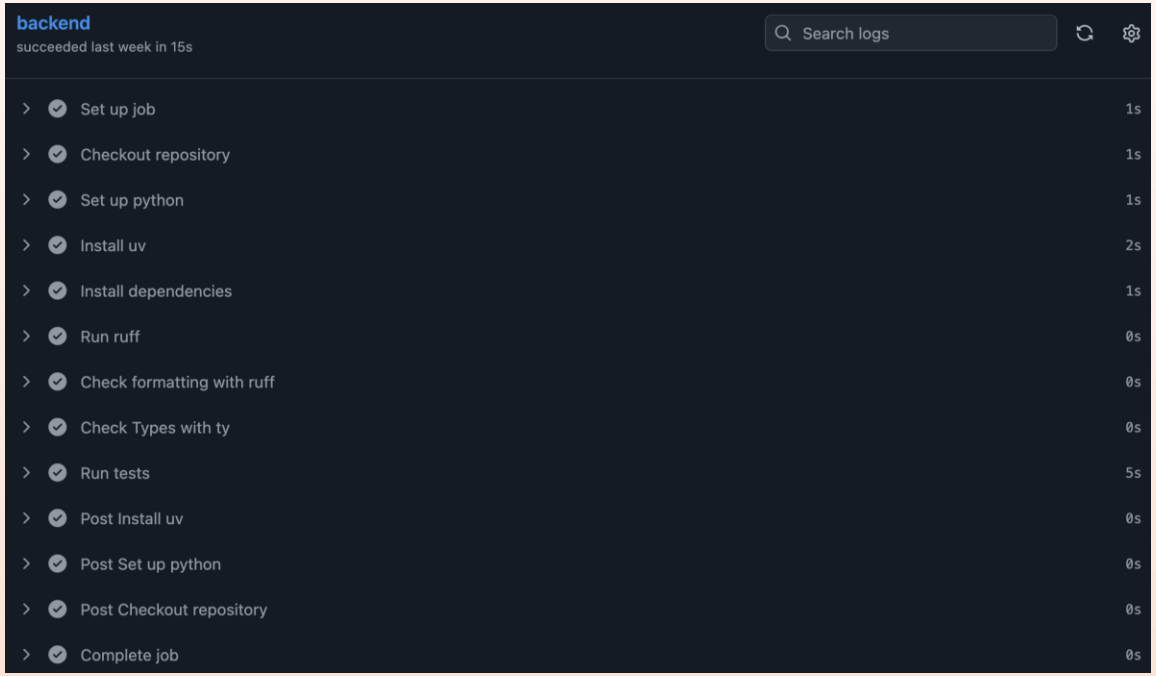
Merge into dev

CI runs again

What the pipeline checks

The jobs check code quality and buildability, then run the backend test suite





Examples covered by the tests

- **Authentication**
- **Document upload validation**
- **Access to manuscripts**

PART 05 / 05

Plans for the second half

The next milestones and the final software

What we have done so far

A focused first version before expanding to harder manuscripts

User

User creation
Main page

A manuscript library

Upload and save scans
Open a page from the library

A recognition baseline

Compared handwriting models
Selected Kraken and PyLaia

Why charters first?

Start with charters

Stronger results on
our test material

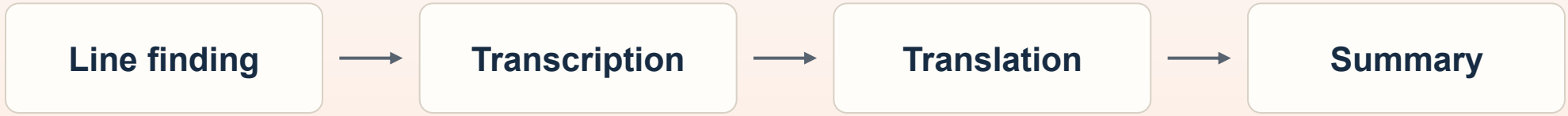
Expand to registers

Dense handwriting needs
further fine-tuning

Priorities for second half

A working pipeline first, then we improve and extend it

First
(baseline)



Fine-tuned line detection
Diverse manuscripts

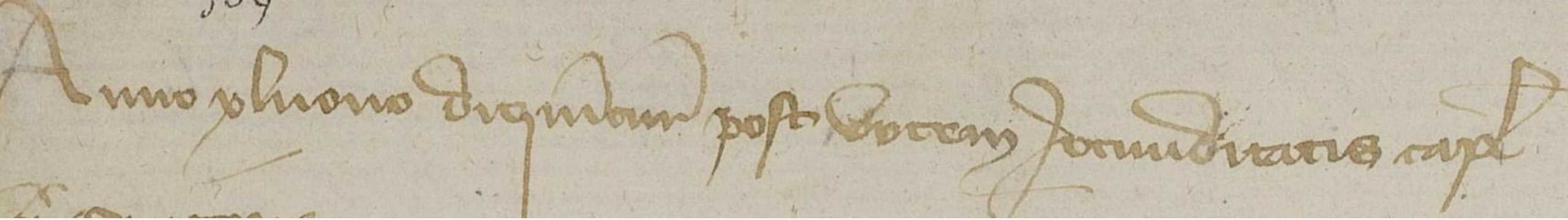
Fine-tuned transcription
Diverse manuscripts

Then

Specialist models
Only where it falls short

If time allows

Classifier
Picks the specialist for you



Thank you for your attention!

Questions & discussion

Project repository

github.com/nlklfor/manulator